Part 2: Testing Analysis – Sidrit Rrustemi

1. Boundary Value Testing

Method tested: DefaultCirculationService.shouldSendReminder(String itemCode, int daysBeforeDue)

In this method, the critical numeric boundary is the daysBeforeDue parameter, which determines whether a reminder should be sent based on the item's due date. Additionally, the derived daysUntilDue (calculated from BorrowRecord.getDueDate()) affects the logic for sending reminders when items are overdue or due today.

I applied **Robust Boundary Value Testing (BVT)** by testing minimum and maximum boundaries (daysBeforeDue = 0 and 7), values just outside the valid range (-1 and 8), typical values (1, 3 and 6), and critical derived boundaries for daysUntilDue (-1, 0, and matching positive values).

TC	daysBeforeDue	daysUntilDue	Expected Result	Notes
BVT_1	-1	3	FALSE	Below lower boundary for daysBeforeDue (invalid input)
BVT_2	0	0	TRUE	Lower boundary matches due today
BVT_3	0	1	FALSE	Lower boundary, not due today
BVT_4	1	1	TRUE	Just above minimum matches due date
BVT_5	3	-1	TRUE	Overdue item, daysUntilDue < 0 triggers reminder
BVT_6	3	3	TRUE	Typical value, exact match
BVT_7	6	6	TRUE	Just below maximum matches due date
BVT_8	7	7	TRUE	Upper boundary matches due date
BVT_9	8	7	FALSE	Above upper boundary for daysBeforeDue (invalid input)I don't

2. Equivalence Class Testing

Method tested: DefaultCirculationService.validateMemberStatus(int memberId)

In this method, I identified equivalence classes based on three key factors: (1) memberId validity, (2) member existence in storage, and (3) combinations of member state attributes including active status, overdue item status, and loan capacity status.

Weak Robust Equivalence Class Testing was used - testing one representative from each valid and invalid class independently.

TC	memberId	Equivalence Class	Expected Result
EC_1	0	Invalid ID (memberId ≤ 0)	"INVALID_MEMBER_ID"
EC_2	1	Member does not exist in storage	"MEMBER_NOT_FOUND"
EC_3	2	All three issues: inactive, has overdue items, at max capacity	"SUSPENDED_MULTIPLE_ISSUES"
EC_4	3	Two issues: inactive and has overdue items	"SUSPENDED_INACTIVE_OVERDUE"
EC_5	4	Two issues: inactive and at max capacity	"SUSPENDED_INACTIVE_CAPACITY"
EC_6	5	Two issues: has overdue items and at max capacity (but active)	"RESTRICTED_OVERDUE_CAPACITY"
EC_7	6	Single issue: inactive member only	"INACTIVE"
EC_8	7	Single issue: has overdue items only (active member)	"OVERDUE_ITEMS"
EC_9	8	Single issue: at max capacity only (active member)	"AT_CAPACITY"
EC_10	9	No issues: active, no overdue items, below capacity	"ACTIVE_GOOD_STANDING"

3. Code Coverage Testing

Method tested: DefaultBorrowPolicy.canBorrow(Member member, int activeBorrows, boolean hasOverdueBooks)

3.1. Statement Coverage

Statement coverage ensures every executable line of code runs at least once during testing.

TC	Member Setup	activeBorrows	hasOverdueBooks	Result
SC_1	valid, active, maxLoans=5	2	FALSE	TRUE

3.2. Branch Coverage

Branch coverage requires testing each decision outcome - both the true and false branches of the return expression must be executed.

TC	Member	activeBorrows	hasOverdueBooks	Result
BC_1	valid, active, maxLoans=5	2	FALSE	TRUE
BC_2	null	2	FALSE	FALSE

3.3. Condition Coverage

Condition coverage mandates that every individual boolean condition evaluates to both true and false at least once across all test cases, independent of overall decision outcomes.

TC	Target Condition	Condition Value	Other Conditions	Result
CC_1	C1	TRUE	C2=T, C3=T, C4=T	TRUE
CC_2	C1	FALSE	not evaluated (short-circuit)	FALSE
CC_3	C2	TRUE	C1=T, C3=T, C4=T	TRUE
CC_4	C2	FALSE	C1=T, C3=T, C4=T	FALSE
CC_5	C3	TRUE	C1=T, C2=T, C4=T	TRUE
CC_6	C3	FALSE	C1=T, C2=T, C4=T	FALSE
CC_7	C4	TRUE	C1=T, C2=T, C3=T	TRUE
CC_8	C4	FALSE	C1=T, C2=T, C3=T	FALSE

3.4. MC/DC Coverage

Modified Condition/Decision Coverage (MC/DC) demonstrates that each condition independently influences the decision outcome. For each condition, two tests must exist where only that condition changes value while others remain fixed, and this change must alter the decision result.

TC	C1	C2	C3	C4	Decision Outcome & Result
DC_1	T	T	T	T	TRUE
DC_2	F	T	T	T	FALSE
DC_3	T	F	T	T	FALSE
DC_4	T	T	F	T	FALSE
DC_5	T	T	T	F	FALSE

Boolean Condition Naming Table used above

ID	Boolean Condition
C1	member != null
C2	member.isActive()
C3	activeBorrows < member.getMembershipType().getMaxLoans()
C4	!hasOverdueBooks